

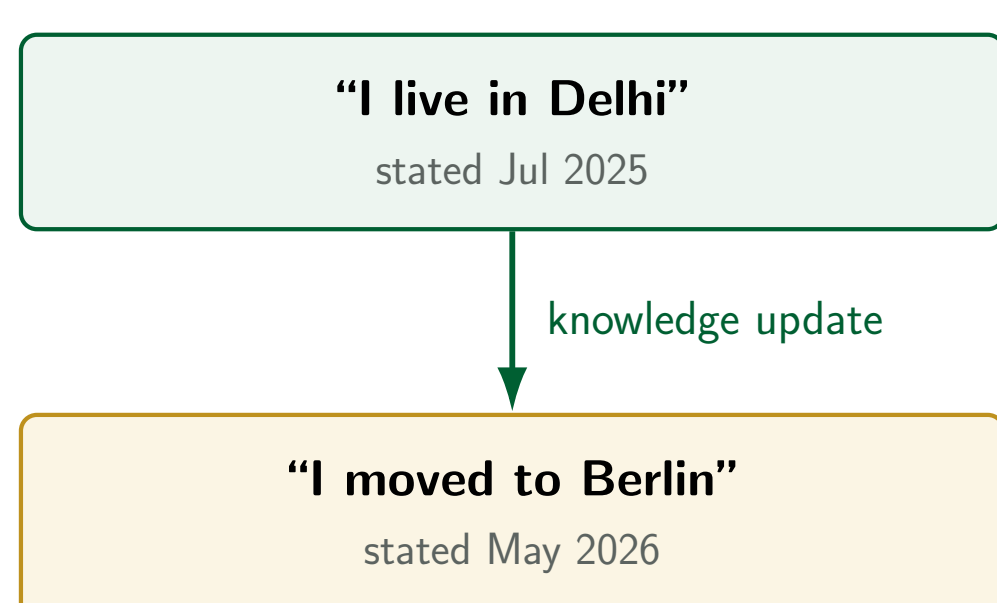
DTSG: A Dynamic Temporal State Graph for Agent Memory

Gungun Jain · Krati Mishra · Sneha
Department of Information Technology · Indira Gandhi Delhi Technical University for Women

An agent’s memory is not a pile of text — it is a set of claims that *change*. DTSG stores **what was said** apart from **what is still true**, and ranks by **relevance** × **validity** × **recency**.

1 The problem

Retrieval-augmented memory ranks by semantic similarity alone. When a fact is updated, the old and the new sentence are *about the same thing*, so they embed almost identically.

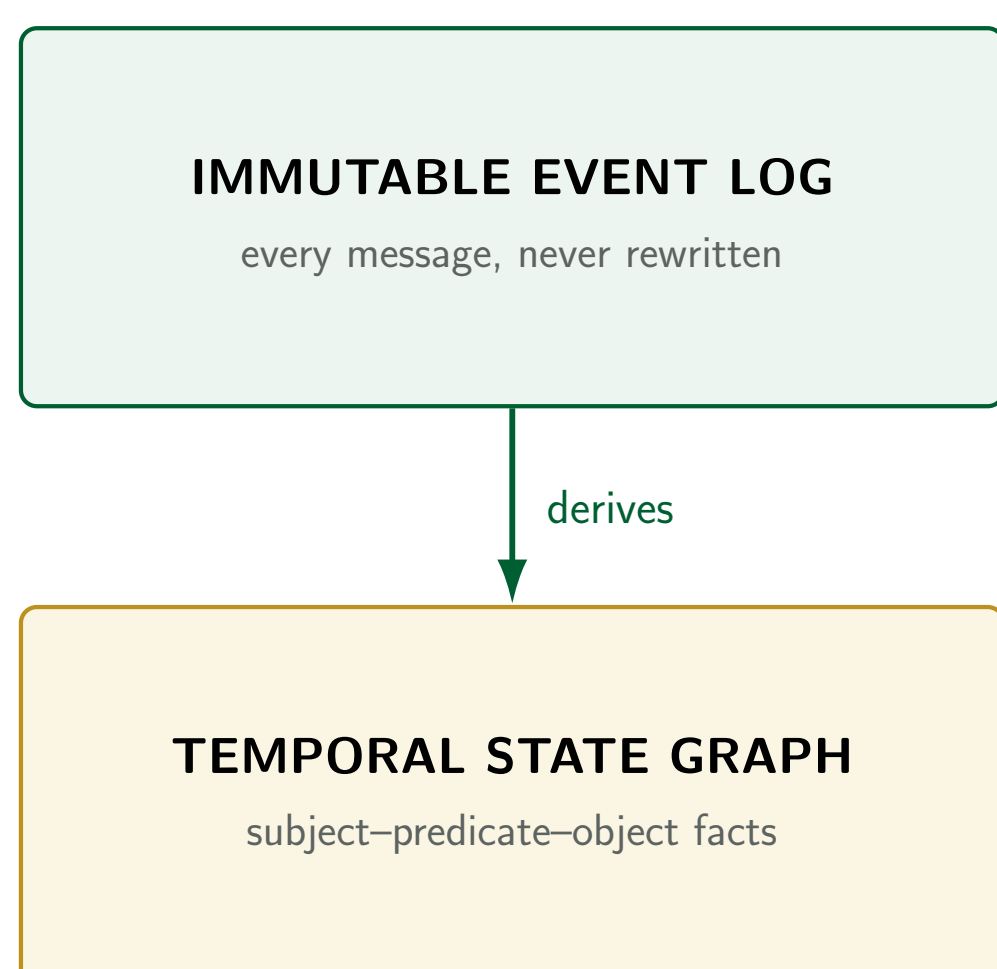


Similarity says what is *relevant*.
It cannot say what is **true now**.

- Which of the two lands on top is decided by noise, not by truth.
- The agent does not hedge — it states the stale answer confidently.
- Deleting the old fact fixes ranking but destroys history.

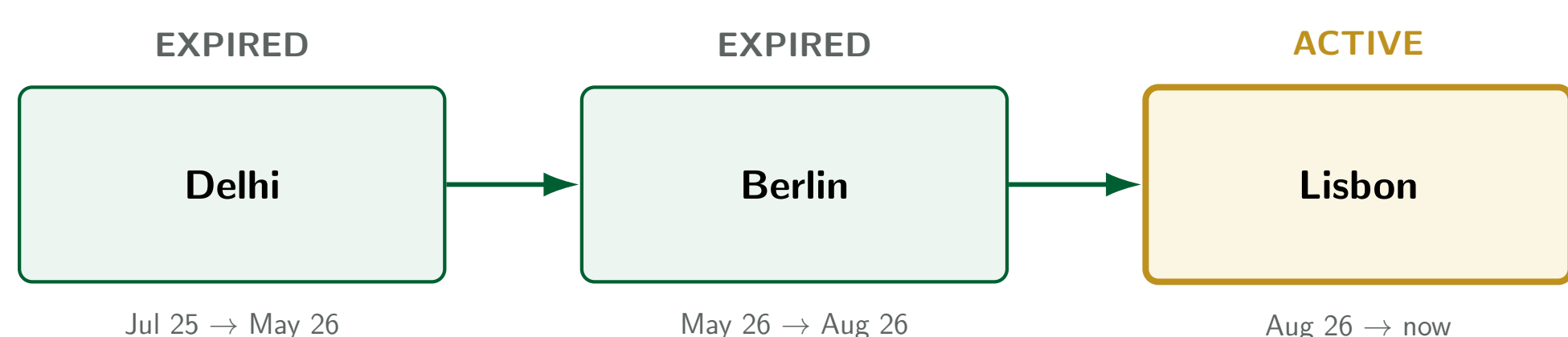
2 The idea

Split memory by mutability: an append-only log of what was said, and a graph of what currently holds.



Each fact carries a **status**, a **validity interval** and a **supersede link**. A contradicted fact is marked **EXPIRED** and pointed at its replacement — never deleted, so history stays queryable.

3 Nothing is deleted



Rows are joined by superseded-by. Two moves produce three rows, not one overwrite — so “*where did I live last spring?*” stays answerable from the same store.

4 Architecture

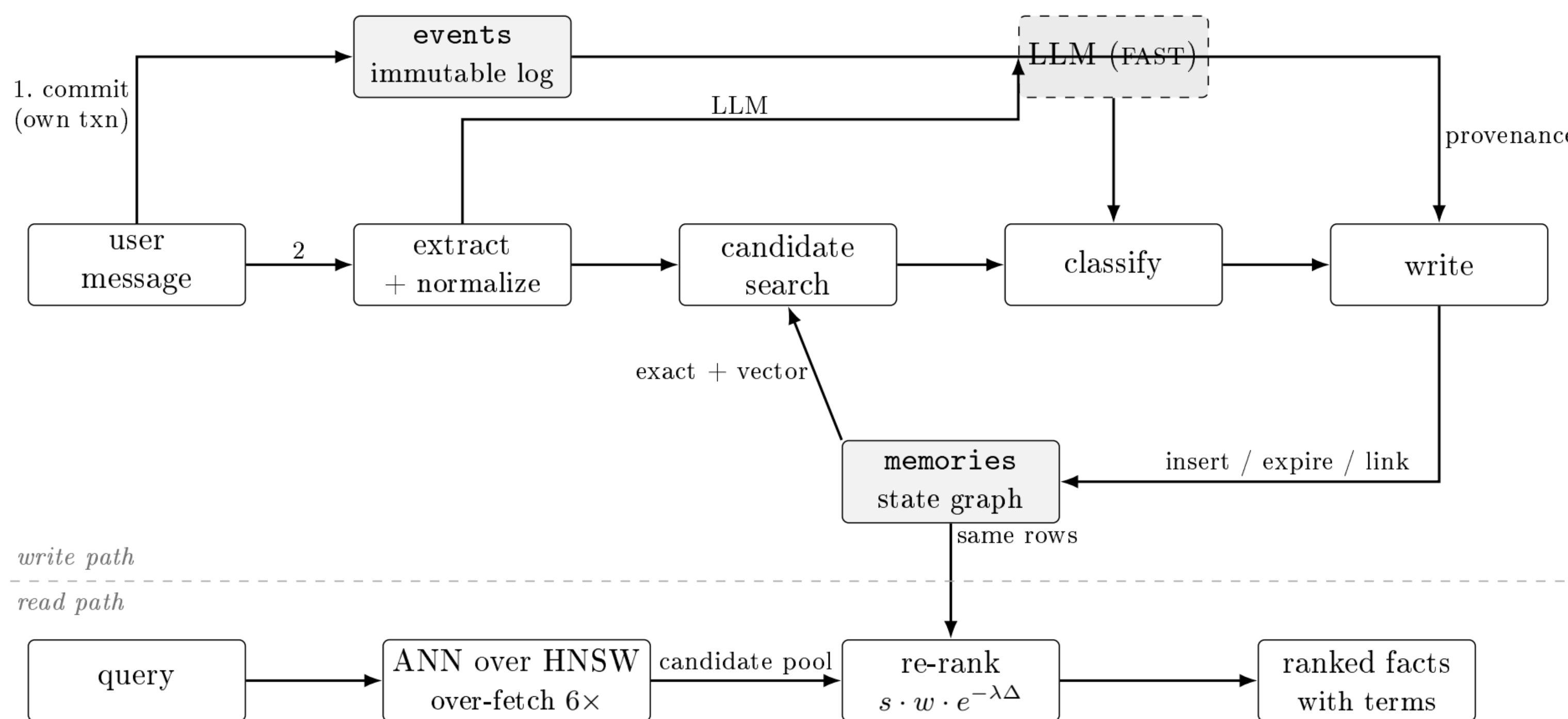


Figure 1. The write path commits the raw message first, in its own transaction, before any derived state. The read path over-fetches from the HNSW index and re-ranks in Python. Dashed nodes are language-model calls.

5 Write path



Normalization folds every first-person subject to user, snake-cases predicates, and maps **22 surface variants** onto **15 canonical predicates** — without it, *lives in* and *resides in* never match and contradictions go undetected.

Candidate search runs two arms: an **exact** match on subject + predicate, always included, and a **vector** arm above a 0.55 similarity floor. The classifier resolves the new fact against them:

REINFORCE	the fact restates a held one — raise its confidence, store no copy
ADDITIVE	the fact coexists — store it, touch nothing
SUPERSEDE	the fact contradicts — store it, expire and link the old

The judgement that matters is **additive vs. supersede** — whether a predicate can hold several values at once. *Speaks* can; *lives in* cannot. Nothing in the schema decides it, so the model is asked, and told to choose the recoverable error when unsure.

6 Read path: temporal ranking

$$S(q, f) = \underbrace{\text{sim}(q, f)}_{\text{relevance}} \times \underbrace{w_{\text{status}}(f, t)}_{\text{validity}} \times \underbrace{e^{-\lambda \Delta t}}_{\text{recency}}$$

relevance
cosine over embeddings

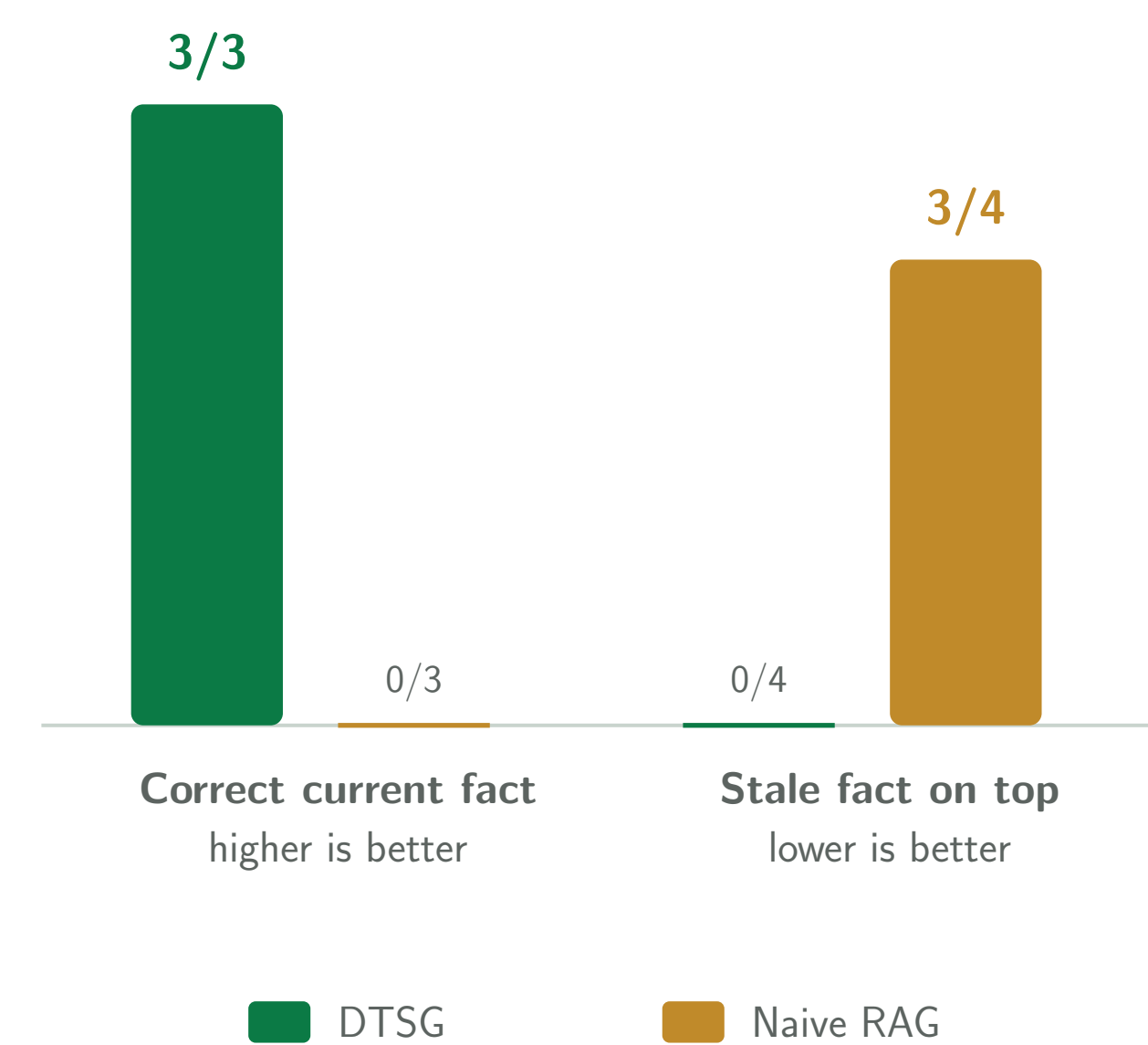
validity
evaluated *at query time*

recency
180-day half-life

The status term is read against the query’s reference time, not from the stored column — so “*where do I live?*” and “*where did I live in March?*” are the same query with a different t , answered from one store.

7 Results

Four hand-built cases in which the *stale* fact is the equal-or-better textual match — the situation where cosine-only retrieval cannot be right by accident. Cases carry their own similarity scores, so no embedding model enters the comparison.



3/3 vs 0/3

DTSG picks the current fact in every scored case;
the cosine-only baseline in none — and never tops a stale fact.

8 Which term does the work?

A controlled scenario: the user moved from Delhi to Berlin 10 days ago and both facts have cosine similarity *exactly* 1.000, so a vector store cannot distinguish them even in principle.

Fact	Status	sim.	score
Berlin	ACTIVE	1.000	0.9622
Delhi	EXPIRED	1.000	0.1443
<i>asked as of 60 days ago, the order inverts:</i>			
Delhi	held then	1.000	1.0000
Berlin	not yet true	1.000	0.1500

With decay switched off the separation still holds — the **status weight**, not recency, is what divides live from dead. And it is a discount, not a filter: raising w_{exp} brings the old fact back for “*where have I lived?*”

9 Conclusion

103

automated tests passing

extraction · normalization · immutability · transactions · chain walking · ranking

Choosing between an old fact and its replacement is a **write-time problem wearing a read-time disguise**: the information needed is the record of which one replaced the other. DTSG writes that record down and spends it at read time.

Next: validate extraction and classification against a live model; evaluate on the knowledge-update subsets of LongMemEval and LoCoMo; learn per-predicate half-lives; entity resolution over objects; multi-hop retrieval.